

PyVRP-Web: Bio-inspired Vehicle Routing Optimization with Interactive Web Visualization

Student ID: 414085193

Department of Software Engineering

Fu Jen Catholic University, Taiwan

Email: 414085193@cloud.fju.edu.tw

Abstract—The Vehicle Routing Problem (VRP) is a combinatorial optimization problem of high practical importance in logistics planning, yet it is NP-hard and infeasible to solve optimally at scale. This paper presents our study of PyVRP, a high-performance open-source VRP solver implementing Hybrid Genetic Search (HGS)—a bio-inspired algorithm combining genetic evolution with local search. We analyze the algorithmic components of HGS in depth, including its SREX crossover operator, population diversity management via Broken Pairs Distance, and adaptive penalty mechanism. Furthermore, we implement PyVRP-Web, a full-stack web application that exposes the HGS solver through an interactive browser interface, enabling zero-code problem definition and real-time geographic route visualization. Benchmark evaluation shows PyVRP achieves a mean gap of 0.22% from best-known solutions on 100 CVRP instances, and 0.40% on 60 VRPTW instances with 1,000 customers. A real-world case study with 16 Taipei convenience stores validates practical usability.

Index Terms—Vehicle Routing Problem, Hybrid Genetic Search, Genetic Algorithm, Bio-inspired Computing, Web Application, Local Search

I. INTRODUCTION

The **Vehicle Routing Problem (VRP)** is a classical combinatorial optimization problem with direct applications in last-mile delivery, public transit, and supply chain management. Given a depot, a fleet of vehicles with limited capacity, and a set of customers with delivery demands, the VRP seeks a minimum-cost set of routes such that every customer is served exactly once and vehicle capacity constraints are satisfied [1]. The VRP with Time Windows (VRPTW) additionally requires each customer to be served within a specified time interval.

VRP is NP-hard: for a problem with merely 20 customers, the number of possible route permutations exceeds 2×10^{18} , making brute-force enumeration computationally infeasible. This motivates the use of **bio-inspired metaheuristics**—algorithms modeled on natural processes such as evolutionary selection and genetic recombination—which trade optimality guarantees for scalability.

This work centers on **PyVRP** [2], a state-of-the-art solver implementing **Hybrid Genetic Search (HGS)**: a framework synergizing Genetic Algorithm (GA) exploration with Local Search exploitation. PyVRP won the 2021 DIMACS VRPTW Challenge and ranked first in the EURO meets NeurIPS 2022 competition.

Our contributions are:

- 1) **Algorithmic Analysis**: A systematic study of HGS components, including SREX crossover [3], broken pairs distance diversity, and adaptive penalty management.
- 2) **System Implementation**: PyVRP-Web, a full-stack web application wrapping HGS in a zero-code interface with real-time map visualization. We identify three concrete barriers preventing practitioners from using research-grade VRP solvers: (i) the requirement to write Python code to assemble the solver model; (ii) the mismatch between PyVRP’s integer planar coordinate system and real-world WGS84 GPS data; and (iii) the absence of visual route output. PyVRP-Web resolves all three.

II. RELATED WORK

A. VRP Solver Landscape

Numerous tools address VRP. **LKH-3** [4] transforms VRP into a TSP and applies the Lin-Kernighan heuristic, but is restricted to academic, non-commercial use. **OR-Tools** [5] from Google uses constraint programming and is easy to use but performs far from the state of the art [2]. **VRPSolver** [6] finds exact solutions but does not scale beyond a few hundred customers. **VROOM** integrates with road network APIs for real-world routing but lacks the solution quality of HGS-based solvers.

PyVRP occupies a unique position: near-optimal performance comparable to specialized C++ solvers, full Python customizability, and availability under the MIT license.

B. Hybrid Genetic Search

HGS was introduced by Vidal et al. [7] for a broad class of VRP variants, maintaining both feasible and infeasible solutions via dynamic penalty coefficients. A key property is that this approach enables exploration of a wider solution space while steering convergence toward feasibility—an adaptive strategy inspired by environmental pressure in natural evolution.

Vidal [8] extended HGS-CVRP with the SWAP* operator and biased fitness criterion for population management. The SREX crossover operator [3] preserves complete sub-routes across generations, avoiding the quality degradation of naive

crossover operators by inheriting intact high-quality route structures.

PyVRP [2] builds on HGS-CVRP by adding VRPTW support, adopting a Python+C++ hybrid architecture for extensibility, and robustifying the implementation while maintaining competitive performance.

III. METHODS

A. Problem Formulation

The **CVRP** is defined on a complete directed graph $G = (V, A)$ where $V = \{0, 1, \dots, n\}$ (0: depot, $1 \dots n$: customers). Customer i has demand $q_i \geq 0$; each vehicle has capacity Q . Objective:

$$\min \sum_{(i,j) \in A} d_{ij} x_{ij} \quad (1)$$

subject to every customer visited exactly once, routes starting/ending at the depot, and total route demand $\leq Q$.

The **VRPTW** additionally requires service at customer i to begin within $[e_i, l_i]$, with service duration s_i and travel time t_{ij} . Vehicles may wait (if arriving early) but cannot violate the late deadline l_i .

B. Hybrid Genetic Search

HGS [2], [7] is the core bio-inspired algorithm. Its key insight is that neither GA nor Local Search alone performs optimally—combining them yields superior results: GA provides *exploration* across the solution space while Local Search provides *exploitation* within promising regions.

The main loop is outlined in Algorithm 1.

Algorithm 1 HGS Main Loop

```

1: Initialize population  $\mathcal{P}$  with  $n_{\min} = 25$  random solutions
2: Set initial penalty weights ( $\alpha = 20, \beta = 6$ )
3: repeat
4:    $(p_1, p_2) \leftarrow \text{TOURNAMENTSELECT}(\mathcal{P})$ 
5:    $c \leftarrow \text{SREX}(p_1, p_2)$ 
6:    $c \leftarrow \text{LOCALSEARCH}(c)$ 
7:   if  $\text{rand}() < p_{\text{repair}}$  and  $\text{infeasible}(c)$  then
8:      $c \leftarrow \text{LOCALSEARCH}(c, \alpha \times 12, \beta \times 12)$ 
9:   end if
10:   $\mathcal{P} \leftarrow \mathcal{P} \cup \{c\}$ 
11:  if  $|\mathcal{P}| > n_{\min} + n_{\text{gen}}$  then
12:     $\text{SURVIVORSELECTION}(\mathcal{P})$ 
13:  end if
14:   $\text{UPDATEPENALTIES}(\mathcal{P})$   $\triangleright$  every 50/100 iters
15: until stopping criterion met
16: return best feasible solution

```

1) *SREX Crossover*: The Selective Route Exchange operator [3] constructs an offspring by (1) inheriting a subset of complete routes from parent B , then (2) greedily inserting unserved customers guided by parent A 's structure. This preserves entire high-quality sub-routes across generations, mimicking chromosomal recombination in biological evolution—where intact functional gene segments are inherited rather than random base-pair mixtures.

2) *Local Search*: Local search [2] accounts for **80–90% of runtime** and is implemented in C++. It operates at two levels:

Node operators evaluate moves between customer pairs (u, v) within the **granular neighbourhood** [9] of size k (CVRP: $k = 20$; VRPTW: $k = 40$), reducing complexity from $O(n^2)$ to $O(kn)$. PyVRP provides 11 operators: (N, M) -exchange for $N \in \{1, 2, 3\}$, $M \leq N$ (implemented via C++ template specialization), MoveTwoClientsReversed, and 2-OPT.

Route operators evaluate cross-route moves: RELOCATE* (best single-customer relocation) and SWAP* (best customer swap with optimal insertion position, enhanced with time-window caching and early termination [2]).

3) *Population Management and Diversity*: The population maintains two sub-populations: *feasible* and *infeasible* solutions. The biased fitness criterion [8] balances quality and diversity—analogueous to natural selection favouring both fitness and genetic diversity within a species:

$$f_{\text{biased}}(s) = r_{\text{qual}}(s) \cdot (1 - \rho) + r_{\text{div}}(s) \cdot \rho \quad (2)$$

where $\rho = n_{\text{elite}}/n_{\min} = 4/25$ and diversity is measured by **Broken Pairs Distance (BPD)**:

$$\text{BPD}(A, B) = \frac{|P_A \Delta P_B|}{2n}, \quad \text{BPD} \in [0, 1] \quad (3)$$

Survivor selection removes duplicates, then eliminates solutions with worst biased fitness until the population returns to $n_{\min} = 25$.

4) *Dynamic Penalty Management*: Constraints are treated as soft via penalized cost:

$$c_{\text{pen}} = d_{\text{total}} + \alpha \Delta_{\text{cap}} + \beta \Delta_{\text{tw}} \quad (4)$$

α and β are dynamically adjusted every 50 iterations to maintain a target feasibility ratio of 43%. This adaptive mechanism acts as an *environmental pressure* in the evolutionary metaphor: when too few solutions are feasible, constraints are tightened; when too many are feasible, constraints are relaxed to encourage exploration. A repair booster multiplies penalties by 12 when attempting to recover infeasible offspring.

TABLE I
KEY HGS PARAMETERS

Parameter	Value	Description
$n_{\min}$ (min_pop_size)	25	Minimum population size
n_{gen} (generation_size)	40	New offspring per generation
n_{elite} (nb_elite)	4	Elite individuals
k nb_granular (CVRP)	20	Neighbour search radius
k nb_granular (VRPTW)	40	Neighbour search radius
target_feasible	43%	Target feasibility ratio
Repair booster	$\times 12$	Penalty multiplier for repair

C. System Architecture: PyVRP-Web

We designed and implemented PyVRP-Web, a four-layer full-stack application (Figure ??):

1) *Presentation*: Vanilla JS + Leaflet.js interactive map

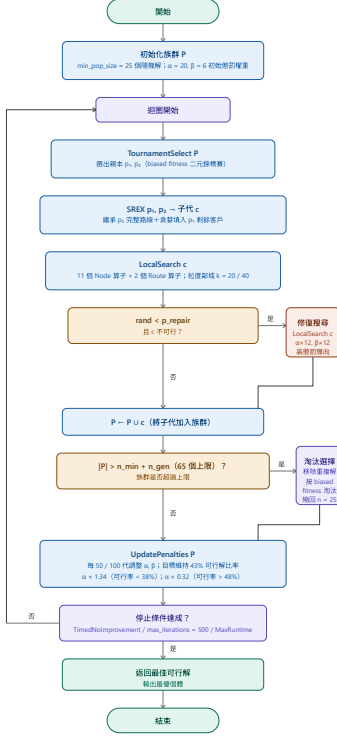


Fig. 1. Main loop of Hybrid Genetic Search (HGS). The algorithm combines genetic crossover (SREX), local search, and dynamic penalty management to explore and exploit the solution space.

- 2) *API*: FastAPI with Pydantic validation and `asyncio.Semaphore(1)` for concurrency control
- 3) *Service*: WGS84→UTM coordinate conversion via `pyproj` (EPSG:32651), neighbourhood tuning ($k = \min(7, n-1)$), per-stop timeline reconstruction
- 4) *Engine*: PyVRP v0.5.0 HGS solver (unmodified black-box)

Design Principle—Zero-Intrusion Encapsulation: PyVRP is treated as an unmodified black-box library. All extensions are encapsulated in surrounding service modules (`coord.py`, `solver.py`, `serializer.py`), ensuring that future PyVRP version upgrades require no changes to the HGS core.

Three key engineering decisions resolved the three practitioner barriers identified in Section I:

(1) **Coordinate Conversion:** PyVRP requires integer planar coordinates. We project WGS84 latitude/longitude to UTM Zone 51N (EPSG:32651) via `pyproj`, then scale by 10 to achieve 0.1-meter integer precision with no metric distortion at Taiwan’s latitude.

(2) **Neighbourhood Tuning:** PyVRP’s default $k = 20$ becomes near-exhaustive for small problems (e.g., 16 customers), causing prohibitively slow local search. We reduce it dynamically to $\min(7, n-1)$, enabling an order-of-magnitude more iterations within the same wall-clock budget.

(3) **Per-Stop Timeline Reconstruction:** PyVRP’s `Route` object provides only the customer visit sequence. We reconstruct arrival and departure times via forward simulation: $t_{arr}^{i+1} = t_{dep}^i + t_{i,i+1}$, where $t_{dep}^i = \max(t_{arr}^i, e_i) + s_i$.

IV. RESULTS

A. Benchmark Performance

We evaluate PyVRP v0.5.0 on two standard benchmark sets following [2].

TABLE II
CVRP BENCHMARK RESULTS (X INSTANCES [10], 100 INSTANCES, 10 SEEDS)

Solver	Mean Cost	Mean Gap	Gap of Mean
BKS	63,106.7	0.00%	0.00%
HGS-CVRP [8]	63,206.1	0.11%	0.16%
HGS-2012 [7]	63,285.8	0.21%	0.28%
PyVRP [2]	63,275.5	0.22%	0.27%

PyVRP achieves a mean gap of **0.22%** from best-known solutions on 100 CVRP instances with 100–1001 customers ($T_{max} = n \times 2.4$ s, 10 seeds). Multiple small-scale instances are solved to **proven optimality** (0% gap), e.g., X-n101-k25, X-n110-k13, X-n115-k10.

TABLE III
VRPTW BENCHMARK RESULTS (H&G 1000-CUSTOMER INSTANCES [11], 60 INSTANCES)

Solver	Mean Cost	Mean Gap	Gap of Mean
BKS	33,143.8	0.00%	0.00%
HGS-DIMACS [12]	33,265.5	0.32%	0.37%
PyVRP [2]	33,296.4	0.40%	0.46%

On 60 VRPTW instances with 1,000 customers (2-hour time limit, 10 seeds), PyVRP achieves **0.40%** mean gap—equivalent to a **2nd-place rank** in the DIMACS VRPTW competition. In extended runs, PyVRP improved **27 of 300** best-known solutions across the full H&G benchmark set. Performance varies by instance type: clustered instances with wide time windows (C2) yield near-optimal results ($\approx 0.04\%$ gap), while random instances with narrow time windows (R1, RC1) are more challenging ($\approx 0.72\%$ gap).

B. System Demonstration

We validate PyVRP-Web on a real-world case study: **Taipei 7-ELEVEN daytime distribution** (Figure 2).

Setup: 1 depot (Unified Enterprise Taipei Distribution Center, 09:30–13:00), 16 stores across Taipei City (time windows 10:00–13:00), 2 large trucks (capacity 140) + 4 medium trucks (capacity 80), average speed 28 km/h.

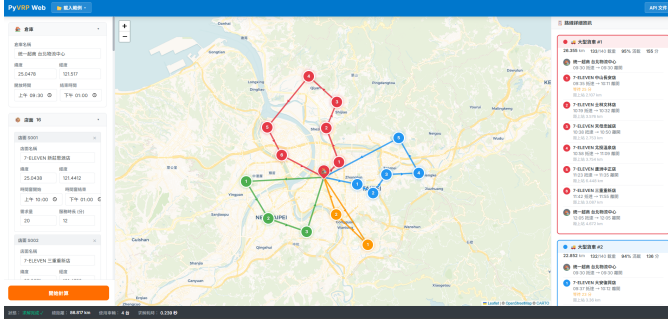


Fig. 2. PyVRP-Web interface showing optimized routes for the Taipei 7-ELEVEN daytime distribution case study (16 stores, 3 routes, ≈ 92 km). Color-coded routes with directional arrows and per-route timelines are shown.

Results: The HGS solver finds a feasible routing plan in approximately **8 seconds** using TimedNoImprovement (max_iterations=500). The solution uses **3 vehicles** with total distance ≈ 92 km. The algorithm naturally clusters geographically proximate stores: northern stores (Shilin, Beitou) in one route, central stores (Da'an, Songshan) in another, and southwestern stores (Banqiao, Wenshan) in a third. This emergent geographic structure arises purely from the HGS objective without explicit geographic instructions—a hallmark of bio-inspired optimization.

The web interface provides: real-time map visualization with directional arrows, per-route timelines (arrival/departure/wait times), and color-coded route highlighting with interactive selection.

V. CONCLUSION

This paper presented a comprehensive study of **Hybrid Genetic Search**—a state-of-the-art bio-inspired algorithm for the Vehicle Routing Problem—and its implementation in PyVRP. We analyzed in depth the biological analogy underlying HGS: SREX crossover mimics chromosomal recombination, biased fitness implements natural selection with a diversity premium to prevent premature convergence, and dynamic penalty management acts as an environmental pressure steering the population toward feasibility.

Our implementation, **PyVRP-Web**, demonstrates that research-grade bio-inspired optimization can be made accessible through modern web technologies. Guided by a zero-intrusion encapsulation principle—leaving PyVRP unmodified and wrapping all extensions in peripheral service modules—the system resolves three practitioner barriers: coordinate system incompatibility, neighbourhood size misconfiguration for small instances, and the absence of per-stop timeline output. The system achieves near-optimal routing solutions within seconds, validated by standard benchmarks (0.22% from BKS on CVRP) and a real-world Taipei distribution scenario.

Future directions include: (i) integrating real road network distances via OSRM for travel time accuracy; (ii) supporting

multi-depot VRP variants; (iii) adding electric vehicle constraints (battery capacity, charging stations); and (iv) visualizing the GA convergence trajectory in real time to enhance educational value.

REFERENCES

- [1] P. Toth and D. Vigo, Eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2014.
- [2] N. A. Wouda, L. Lan, and W. Kool, “PyVRP: a high-performance VRP solver package,” *INFORMS Journal on Computing*, vol. 36, no. 4, pp. 943–955, 2024, arXiv:2403.13795.
- [3] Y. Nagata and S. Kobayashi, “A memetic algorithm for the pickup and delivery problem with time windows using selective route exchange crossover,” in *Parallel Problem Solving from Nature – PPSN XI*. Springer, 2010, pp. 536–545.
- [4] K. Helsgaun, “An extension of the Lin-Kernighan-Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems,” Roskilde University, Tech. Rep., 2017.
- [5] L. Perron and V. Furnon, “OR-Tools,” Google LLC, 2022. [Online]. Available: <https://developers.google.com/optimization/>
- [6] A. Pessoa, R. Sadykov, E. Uchoa, and F. Vanderbeck, “A generic exact solver for vehicle routing and related problems,” *Mathematical Programming*, vol. 183, pp. 483–523, 2020.
- [7] T. Vidal, T. G. Crainic, M. Gendreau, and C. Prins, “A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows,” *Computers & Operations Research*, vol. 40, no. 1, pp. 475–489, 2013.
- [8] T. Vidal, “Hybrid genetic search for the CVRP: open-source implementation and SWAP* neighborhood,” *Computers & Operations Research*, vol. 140, p. 105643, 2022.
- [9] P. Toth and D. Vigo, “The granular tabu search and its application to the vehicle-routing problem,” *INFORMS Journal on Computing*, vol. 15, no. 4, pp. 333–346, 2003.
- [10] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, “New benchmark instances for the capacitated vehicle routing problem,” *European Journal of Operational Research*, vol. 257, no. 3, pp. 845–858, 2017.
- [11] J. Homberger and H. Gehring, “Two evolutionary metaheuristics for the vehicle routing problem with time windows,” *INFOR: Information Systems and Operational Research*, vol. 37, no. 3, pp. 297–318, 1999.
- [12] W. Kool, J. O. Juninck, E. Roos, K. Cornelissen, P. Agterberg, J. van Hoorn, and T. Visser, “Hybrid genetic search for the vehicle routing problem with time windows: a high-performance implementation,” OR-TEC, Tech. Rep., 2022.